

Movie Watchlist — Bulut Mimarilerinde Test Mühendisliği Dönem Projesi Final Raporu

Öğrenciler: Çağdaş Karataş, Eren Şeremet

Ders: Bulut Mimarilerinde Test Mühendisliği

Tarih: Mayıs 2026

1. Giriş

Bu proje, modern bir bulut-native uygulama geliştirme ve test sürecinin tüm katmanlarını kapsamaktadır. Seçilen konu **Movie Watchlist** (film izleme listesi servisi) olup kullanıcıların film ekleyip izleme durumlarını takip edebildiği bir REST API ve basit bir web arayüzü sunmaktadır.

Proje konu olarak Movie Watchlist seçilmiştir; çünkü domain yalındır, CRUD işlemlerinin tamamını barındırır ve test piramidinin her katmanı için anlamlı senaryolar üretmeye elverişlidir.

2. Mimari

Uygulama üç ana bileşenden oluşmaktadır:

FastAPI (Backend)

Python 3.12 + FastAPI 0.115 ile yazılmış RESTful API. SQLAlchemy ORM aracılığıyla SQLite (geliştirme) veya PostgreSQL (üretim/entegrasyon test) ile iletişim kurar. 8 endpoint bulunmaktadır: GET /health, POST /movies, GET /movies, GET /movies/search, GET /movies/{id}, PATCH /movies/{id}, DELETE /movies/{id}, GET /stats.

Streamlit (UI)

Hızlı prototipleme amacıyla tercih edilmiştir. Sidebar'da film ekleme formu, ana ekranda arama ve liste bulunmaktadır. E2E testler bu arayüz üzerinden Playwright ile çalıştırılmaktadır.

PostgreSQL

Üretim ve entegrasyon test ortamında kullanılmaktadır. Docker ve Kubernetes ortamlarında ayrı bir servis olarak çalışır.

Katmanlar Docker Compose veya Kubernetes üzerinde birlikte ayağa kaldırılabilir.

3. Test Stratejisi

Proje, klasik test piramidi modelini uygulamaktadır:

3.1 Unit Testler (23 test)

tests/unit/ altında iki dosya bulunmaktadır:

- **test_movie_service.py** (14 test): Service katmanı fonksiyonları in-memory SQLite üzerinde test edilmiştir. create_movie, get_movies, update_movie, delete_movie, search_movies, get_stats fonksiyonlarının tüm dalları kapsanmıştır.
- **test_endpoints.py** (9 test): FastAPI TestClient ile HTTP endpoint'leri test edilmiştir. 201, 204, 404, 422 durum kodları doğrulanmıştır.

factory-boy kütüphanesi ile MovieFactory tanımlanmış, test verisi üretimi deterministik hale getirilmiştir.

Coverage: %94 (hedef %70)

3.2 Integration Testler (3 test)

tests/integration/ altında testcontainers kütüphanesi kullanılarak gerçek bir PostgreSQL 16 container'ı ayağa kaldırılmış ve ORM sorguları bu container'a karşı doğrulanmıştır:

1. Film oluşturma ve geri okuma
2. watched filtresi ile filtreleme
3. Güncellenmenin yeni session'da kalıcı olması

Her test öncesinde TRUNCATE TABLE ile veri temizlenmiştir.

3.3 E2E Testler (4 test)

tests/e2e/ altında Playwright Chromium (headless) ile Streamlit UI üzerinden senaryolar çalıştırılmıştır:

1. **test_add_movie_appears_in_list**: Form üzerinden film ekleme, listenin güncellenmesi
2. **test_mark_movie_as_watched_updates_stats**: Film izlendi işaretlendiğinde UI'da ✓ görünmesi
3. **test_search_filters_movies**: Arama kutusunun yalnızca eşleşen filmleri göstermesi
4. **test_delete_removes_movie**: Silinen filmin listeden kaldırılması

Streamlit'in WebSocket tabanlı rerun mekanizmasının headless modda tutarsız çalışması nedeniyle, yazma işlemleri API üzerinden yapılmış; UI'nın doğru render ettiği doğrulanmıştır.

3.4 Postman / Newman (5 istek)

postman/collection.json içinde 5 istek ve toplam 10 test assertion bulunmaktadır. newman run ile CI pipeline'da smoke test olarak çalıştırılmaktadır.

4. Pipeline ve Deploy

4.1 GitHub Actions

.github/workflows/ci.yml içinde 6 job tanımlıdır:

Job

Görevi

lint	ruff check ile kod kalitesi
test	Unit testler + coverage raporu (artifact)
integration	Testcontainers ile PostgreSQL entegrasyon testleri
docker-build	Multi-stage image build + tar artifact
newman-smoke	Docker container ayağa kaldırılır, newman çalıştırılır
k8s-validate	kubeconform ile manifest şema doğrulaması

4.2 Docker

Dockerfile iki aşamalıdır:

- **builder:** python:3.11-slim + gcc + libpq-dev → bağımlılıkları ~/.local'e yükler
- **runtime:** Yalnızca libpq5 ve curl içerir, builder'dan kopyalanır

Sonuç image boyutu: ~73 MB (tek aşamalı yaklaşıma kıyasla önemli ölçüde küçük).

4.3 Kubernetes (Minikube)

k8s/ dizininde 4 manifest bulunmaktadır:

- configmap.yaml: Uygulama konfigürasyonu ve veritabanı bağlantı bilgisi (Secret)
- postgres.yaml: PostgreSQL Deployment + ClusterIP Service
- deployment.yaml: 2 replica, liveness/readiness probe, kaynak limitleri
- service.yaml: NodePort (30080)

imagePullPolicy: Never ile Minikube'ün yerel image cache'i kullanılmaktadır.

5. Performans ve Gözlemlenebilirlik

Bu projede Prometheus/Grafana entegrasyonu ve yük testi (Locust/k6) kapsam dışında bırakılmıştır. Gelecekte eklenebilecek iyileştirmeler:

- FastAPI'ye prometheus-fastapi-instrumentator eklenerek /metrics endpoint'i açılabilir
- Grafana dashboard'u ile istek sayısı, gecikme ve hata oranı izlenebilir
- locust ile 100 eşzamanlı kullanıcı senaryosu oluşturulabilir

6. Sonuç ve Öğrendiklerim

Proje, test piramidinin tüm katmanlarını (unit, integration, E2E, API) kapsayan, CI/CD pipeline'a sahip, container ve Kubernetes üzerinde çalışan bir uygulama ortaya koymuştur.

Sayısal Özet:

Metrik

Değer

Toplam test sayısı	35 (23 unit + 3 integration + 4 E2E + 5 Newman)
Code coverage	%94
Docker image boyutu	~73 MB
CI job sayısı	6
K8s replica sayısı	2

Zorluklar:

- **Minikube image cache:** `eval $(minikube docker-env)` ile doğru Docker context'ine geçilmezse image bulunamaz hatası alınır.
 - **Streamlit + Playwright:** Streamlit'in WebSocket tabanlı rerun mekanizması, headless Chromium'da standalone butonların API çağrısı yapmamasına neden oldu. Yazma işlemleri direkt API üzerinden yapılarak aşıldı.
 - **psycpg2-binary:** Python 3.12 için wheel'in kurulabilmesi için `pip install --force-reinstall` gerekmektedir.
 - **In-memory SQLite + SQLAlchemy:** Birden fazla bağlantının aynı in-memory DB'yi paylaşması için `StaticPool` kullanılmalıdır.
-

7. Kaynaklar

- FastAPI Docs: <https://fastapi.tiangolo.com>
- Testcontainers Python: <https://testcontainers-python.readthedocs.io>
- Playwright Python: <https://playwright.dev/python>
- SQLAlchemy 2.0: <https://docs.sqlalchemy.org>
- Kubeconform: <https://github.com/yannh/kubeconform>
- GitHub Actions: <https://docs.github.com/en/actions>